

Apunte de repaso: Posterior Collapse y el rol de la discretización

Lectura estimada: ~1 hora. Pensado para un repaso "por encima" antes de profundizar mañana.

Este apunte junta dos hilos de tu curso que a primera vista parecen distintos, pero que terminan conectados por una misma idea: cuando un espacio latente es continuo y "blando", el modelo tiene un camino fácil para volverse perezoso (ignorar el latente); cuando el espacio es discreto y "duro", ese camino se cierra. Vamos a ver esto primero en VAEs (posterior collapse) y después en síntesis de programas (de LPN a NLI).

PARTE 1 — Posterior Collapse en VAEs

1.1 Repaso mínimo: ¿qué es un VAE?

(Basado en: *An Introduction to Variational Autoencoders*, Kingma & Welling, y usado como marco común por los otros dos papers de esta parte).

Un VAE quiere describir datos de alta dimensión x (imágenes, texto, audio) usando variables latentes z de baja dimensión. La receta tiene tres piezas:

- Un **prior** $p(z)$ — usualmente una Gaussiana estándar — que dice qué formas de z son "normales" a priori.
- Un **encoder** $q_\phi(z|x)$ que, dado un dato x , propone una distribución sobre z (el "posterior aproximado").
- Un **decoder** $p_\theta(x|z)$ que, dado un z , reconstruye x .

Como no podemos calcular el posterior verdadero $p(z|x)$ exactamente, entrenamos maximizando una cota inferior llamada **ELBO** (Evidence Lower BOund):

$$\text{ELBO}(x) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$$

El primer término es "qué tan bien reconstruyo x "; el segundo es una penalización de **divergencia KL** que castiga a $q_\phi(z|x)$ por alejarse del prior $p(z)$. Intuitivamente, la KL es una medida de "distancia" entre dos distribuciones: si son idénticas, $\text{KL} = 0$.

Este segundo término es la semilla de todo lo que viene.

1.2 ¿Qué es el "posterior collapse"?

El posterior collapse ocurre cuando, al entrenar, el encoder simplemente copia el prior:

$$q_\phi(z|x) = p(z) \quad \text{para todo } x$$

Es decir, **el latente deja de depender de x** . El encoder "se rinde": en vez de codificar información útil sobre el dato, produce siempre la misma distribución (la del prior), sin importar qué dato entra. El resultado es un VAE que reconstruye razonablemente bien (si el decoder es suficientemente potente por sí solo) pero cuyo espacio latente es inútil: no sirve para representar, comprimir ni razonar sobre los datos.

Los dos papers de teoría que leíste dan dos explicaciones distintas — y complementarias — de por qué pasa esto.

1.3 Explicación 1: Posterior collapse como transición de fase

(Basado en: *Posterior Collapse as a Phase Transition in Variational Autoencoders*, Li, Zhang, Zhang & Chen, 2025).

Este paper mira el problema con ojos de **física estadística**. La idea de una "transición de fase" (concepto que quizás conozcas del agua congelándose): un sistema cambia abruptamente de un régimen a otro cuando un parámetro cruza un valor crítico, y esa transición suele marcarse por una discontinuidad en alguna cantidad física. Aquí, la "cantidad física" es la divergencia KL.

Idea central. Los autores toman el VAE Gaussiano estándar, donde:

$p(x|z) \sim \text{Normal}(f(z), \sigma'^2)$ (el decoder reconstruye con ruido σ'^2)
 $q(z|x) \sim \text{Normal}(\mu(x), \sigma(x)^2)$ (el encoder)
 $p(z) \sim \text{Normal}(\theta, 1)$ (prior estándar)

y estudian los **puntos extremos** (máximos/mínimos) de la función objetivo promediada sobre todos los datos, L_{VAE} . Usando cálculo variacional (básicamente, "derivar" respecto a funciones enteras en vez de respecto a números) encuentran que hay **múltiples soluciones extremas posibles**, y una de ellas es exactamente el posterior collapse:

$f(z) = \langle x \rangle$ (el decoder ignora z y siempre predice el promedio de los datos)
 $\mu(x) = \theta$
 $\sigma(x) = 1 \rightarrow$ esto implica $q(z|x) = p(z)$ (colapso)

Es decir: **el colapso no es un error de entrenamiento, es una solución matemáticamente válida** (un extremo legítimo) de la ecuación del ELBO. La pregunta entonces no es "¿por qué el modelo se equivoca?" sino "¿quién gana la competencia entre esta solución trivial y las soluciones no triviales (que sí usan el latente)?"

El criterio crítico. Para saber cuál solución gana, los autores analizan la **estabilidad** del punto de colapso (técnicamente, si la segunda derivada — el Hessiano — es negativa definida ahí, el punto es un máximo local estable; si no, el modelo puede "escaparse" hacia una mejor solución). El resultado es sorprendentemente simple — un criterio que depende solo de dos cosas: el hiperparámetro de ruido del decoder σ'^2 y la estructura de los datos, medida por los **autovalores de la matriz de covarianza** de los datos, $\xi_1^2, \dots, \xi_N^2$ (piensa en esto como "cuánta varianza real tiene cada dirección principal de tus datos", vía PCA):

Colapso ocurre $\Leftrightarrow \sigma'^2 > \max(\xi_1^2, \xi_2^2, \dots, \xi_N^2)$

En palabras simples: si le permites al decoder asumir que hay *mucho* ruido en la reconstrucción (σ'^2 grande), entonces ya no vale la pena que el encoder se esfuerce en codificar la señal real de los datos — el "beneficio" de usar z para reconstruir mejor no compensa el "costo" en KL. Cuando el ruido asumido supera la varianza real más grande que tienen los datos, el óptimo es simplemente rendirse. Los autores validan esto con datos sintéticos y con CIFAR-10, encontrando el punto crítico exacto en ambos casos, y muestran que la KL diverge en su primera y segunda derivada justo en ese punto crítico — la firma matemática de una transición de fase.

(Nota de contexto: σ'^2 juega un rol análogo al parámetro β del conocido β -VAE, que multiplica el término de KL — subir β o subir σ'^2 tienen el mismo efecto de empujar al modelo hacia el colapso.)

1.4 Explicación 2: Posterior collapse como no-identificabilidad

(Basado en: *Posterior Collapse and Latent Variable Non-identifiability*, Wang, Blei & Cunningham, NeurIPS 2021).

Este paper da una explicación distinta y muy elegante, basada en un concepto clásico de estadística: la **identificabilidad**.

Concepto clave (identificabilidad). Una variable latente z es "no identificable" cuando la verosimilitud de los datos **no cambia** según el valor de z que uses:

$$p(x \mid z = \bar{z}_0; \theta) = p(x \mid z = \bar{z}; \theta) \quad \text{para cualquier par } \bar{z}_0, \bar{z}$$

En palabras: si probaras dos valores distintos de z y el decoder produjera *exactamente la misma* distribución sobre x en ambos casos, entonces observar x no te da ninguna pista sobre cuál z lo generó. Es como preguntar "¿qué llave abrió la puerta?" cuando todas las llaves del llavero abren la misma puerta: la pregunta no tiene respuesta informativa.

El **teorema central del paper** dice, ni más ni menos, que esto es *exactamente* lo mismo que el posterior collapse:

$$\text{El posterior colapsa} \iff \text{el latente } z \text{ es no identificable}$$

La demostración es corta y usa solo la regla de Bayes:

$$p(z|x) \propto p(z) \cdot p(x|z)$$

Si z es no identificable, $p(x|z)$ es constante en z (no depende de z), entonces $p(z|x) \propto p(z) \cdot \text{constante} \propto p(z)$, es decir, el posterior es proporcional al prior — y como ambos son densidades, deben ser iguales. El argumento inverso (colapso $\Rightarrow$ no identificable) se hace igual de directo.

Por qué esto importa tanto. Esta equivalencia implica algo fuerte: **el posterior collapse no es un problema de redes neuronales, ni de inferencia variacional, ni de optimización** — es una propiedad del *modelo generativo y los datos*, punto. Los autores lo demuestran con modelos clásicos sin redes neuronales y sin aproximación variacional (mezclas Gaussianas, PPCA con inferencia exacta vía HMC) y el colapso aparece igual cuando el modelo hace que la variable latente sea no identificable para esos datos.

Un ejemplo concreto e ilustrativo del paper: ajustan un modelo de mezcla Gaussiana con $K=2$ clusters. Hay dos soluciones que maximizan la verosimilitud por igual: (1) una donde los dos clusters aprendidos son distintos entre sí — ahí, saber a qué cluster pertenece un punto sí cambia la verosimilitud, entonces z es identificable y el posterior no colapsa; (2) otra donde *ambos* clusters aprendidos terminan siendo la misma distribución (la marginal de los datos) — ahí, no importa qué cluster asignes, la verosimilitud es igual, z es no identificable, y el posterior colapsa exactamente al prior. **Mismo modelo, mismos datos, dos óptimos igual de buenos en verosimilitud — pero uno usa el latente y el otro no.**

La solución que proponen (LIDVAE). Si el problema es que la función $p(x|z;\theta)$ no es inyectiva (uno-a-uno) en z , la solución directa es forzar que sea inyectiva. Construyen un decoder a partir de **mapas de Brenier** (funciones que son el gradiente de una función convexa, garantizadas biyectivas — piensa en ellas como "deformaciones" del espacio que nunca pliegan ni superponen puntos) parametrizados con redes neuronales convexas (ICNN). Con esto, matemáticamente **garantizan** que z sea identificable para *cualquier* dataset, sin tocar el objetivo de entrenamiento ni usar trucos de optimización — y muestran empíricamente que evita el colapso mejor que β -VAE o el "lagging encoder", manteniendo un ajuste (log-likelihood) competitivo.

1.5 ¿Por qué la discretización de VQ-VAE ayuda? La conexión con "Discrete Representation Learning"

(Basado en: **Neural Discrete Representation Learning** — el paper del VQ-VAE — van den Oord, Vinyals & Kavukcuoglu, NeurIPS 2017. Este es el paper que leíste la semana pasada).

El VQ-VAE cambia dos cosas respecto a un VAE normal: (1) el encoder produce códigos **discretos**, no continuos; (2) el prior se **aprende** después de entrenar, en vez de ser fijo. El mecanismo:

- Hay un **codebook** (diccionario) de K vectores aprendibles $e_1, \dots, e_K \in \mathbb{R}^D$.
- El encoder produce un vector continuo $z_e(x)$.
- Se reemplaza $z_e(x)$ por su **vecino más cercano** en el codebook:

$$z_q(x) = e_k, \quad \text{donde } k = \operatorname{argmin}_j ||z_e(x) - e_j||^2$$

- El posterior $q(z=k|x)$ es **determinista** (probabilidad 1 en el vecino más cercano, 0 en todo lo demás) y, como el prior sobre z se toma **uniforme** sobre los K códigos durante el entrenamiento, la divergencia KL es una **constante exacta**: $KL = \log K$.

Esto es la clave. Fijate en la fórmula del ELBO de la sección 1.1: si el término de KL es una constante, ya no depende de los parámetros del encoder — su gradiente respecto a esos parámetros es cero. **No hay ninguna presión de entrenamiento empujando a $q(z|x)$ hacia el prior**, porque matemáticamente ya no hay "distancia" que reducir; siempre vale $\log K$. El único gradiente que sí le llega al encoder viene enteramente del error de reconstrucción, así que la única forma de mejorar es *usar* el latente para reconstruir mejor. Los propios autores lo resumen así: usar VQ "permite al modelo evitar problemas de posterior collapse... típicamente observados en el marco VAE" — y lo demuestran de forma dramática con un experimento de dos etapas sobre DeepMind Lab donde apilan un decoder autorregresivo muy potente (justo la situación que normalmente rompe a los VAEs continuos, porque un decoder potente puede ignorar el latente y modelar x por sí solo) y el VQ-VAE sigue usando sus 3 latentes de forma significativa.

Conectando esto con las dos teorías de arriba (síntesis propia, no está en el paper original):

- **Desde la óptica de la transición de fase (1.3):** en un VAE Gaussiano, hay un "dial" continuo (σ'^2 , la varianza de reconstrucción) que, si se pasa de cierto umbral, hace que colapsar sea la solución óptima. En VQ-VAE **ese dial no existe** en la misma forma: la cuantización es una operación dura (nearest-neighbor), no un ruido Gaussiano ajustable que pueda "tragarse" la señal. No hay una varianza que crecer gradualmente hasta cruzar un umbral crítico — el mecanismo entero que produce la transición de fase en el caso Gaussiano no está presente.
- **Desde la óptica de la no-identificabilidad (1.4):** el colapso ocurre si y solo si $p(x|z)$ no depende de z . Con un codebook finito y bien entrenado, cada código e_k corresponde a una región distinta del espacio de entrada — si dos códigos distintos produjeran siempre la misma reconstrucción, ese código sería redundante y el error de reconstrucción no mejoraría al usarlo (de hecho, esto es justo lo que causa "codebook collapse", un problema *distinto* pero relacionado, donde algunas entradas del codebook simplemente no se usan nunca). Mientras el codebook se mantenga diverso, la función decoder es, en la práctica, "identificable por construcción": cada código activo corresponde a una reconstrucción distinguible.

En otras palabras: **la discretización no resuelve el posterior collapse por accidente — ataca directamente el mecanismo que las dos teorías identificaron como la causa raíz:** elimina el canal continuo por el que la KL podía presionar hacia el colapso, y estructuralmente favorece que la función $z \rightarrow \hat{x}$ sea informativa (identificable) en vez de constante.

PARTE 2 — De LPN a NLI: por qué discretizar la síntesis de programas

2.1 El problema: aprender programas a partir de ejemplos

Ambos papers atacan "**programming by example**" (PBE): dado un conjunto de pares entrada-salida $\{(x_i, y_i)\}$ generados por un programa oculto p , y una nueva entrada x_{n+1} , predecir $p(x_{n+1})$ — sin que el modelo vea jamás el programa mismo, solo ejemplos de su comportamiento. La meta no es memorizar los ejemplos sino **inducir la lógica** del programa para generalizar a casos nuevos, incluyendo casos **fuera de distribución** (OOD): programas más largos, combinaciones de operaciones nunca vistas juntas, etc.

2.2 LPN — Latent Program Network: un espacio latente continuo

(Basado en: *Searching Latent Program Spaces*, Macfarlane & Bonnet, NeurIPS 2025 — el paper que leíste hace dos semanas, cronológicamente el primero de los dos).

LPN combina tres piezas, todas diferenciables de punta a punta:

1. **Encoder** $q_\phi(z|x,y)$: mapea cada par entrada-salida a una distribución Gaussiana en un espacio latente **continuo** z . Los latentes de distintos pares de una misma especificación se agregan por **promedio** (esto los hace invariantes al orden de los pares).
2. **Decoder** $p_\theta(y|x,z)$: dado un z (el "programa implícito") y una entrada x , predice la salida.
3. **Optimización del latente en tiempo de prueba (test-time search)**: como el encoder puede fallar en dar el mejor z (sobre todo en tareas nuevas), se refina con **ascenso de gradiente** directamente sobre la verosimilitud del decoder:

$$z' = z + \alpha \cdot \nabla_z \sum_i \log p_\theta(y_i|x_i, z)$$

Esto es posible porque el decoder es una red diferenciable: buscar el mejor "programa" se vuelve un problema de optimización continua en vez de una búsqueda combinatoria discreta — de ahí el nombre del paper. Los resultados son fuertes: en ARC-AGI, activar la búsqueda por gradiente **duplica** el rendimiento fuera de distribución respecto a no usarla.

La limitación que los propios autores señalan. En su sección de conclusiones, escriben literalmente que LPN "ha mostrado tener capacidades limitadas para la generalización composicional" (*limited abilities for compositional generalization*). Y en el apéndice muestran ejemplos concretos: si entrenan LPN viendo la operación "extraer patrón" y la operación "pintar de rosa" por separado, y a la hora de la prueba le piden **componer ambas** (extraer y luego pintar), LPN falla parcialmente — la búsqueda por gradiente ayuda a veces, pero con 3 operaciones compuestas la búsqueda ya no logra corregir el error. La razón intuitiva: z es un único vector continuo, "monolítico" — no hay una noción natural de "una parte del vector es la operación A y otra parte es la operación B" que se pueda recombinar de forma sistemática.

Y lo más importante para tu pregunta: en su sección de **trabajo futuro**, LPN dice explícitamente que una dirección prometedora es "explorar representaciones latentes discretas para mejorar la generalización composicional" — **esta frase es literalmente la semilla del segundo paper**.

2.3 NLI — Neural Language Interpreter: discretizar el programa

(Basado en: **Gradient-Based Program Synthesis with Neurally Interpreted Languages**, Macfarlane, Bonnet, van Hoof & Lelis, ICLR 2026 — el paper que leíste la semana pasada, y que de hecho cita a LPN como su predecesor directo).

NLI toma la idea de LPN pero cambia la naturaleza del latente: en vez de un único vector continuo, el programa es una **secuencia de tokens discretos** $z = (z_1, \dots, z_T)$, cada uno tomado de un **codebook** aprendido de tamaño K (igual en espíritu al codebook de VQ-VAE que ya conoces de la Parte 1). La arquitectura tiene dos piezas nuevas clave:

(a) El **inductor (encoder)** produce tokens discretos vía **Gumbel-Softmax**. Necesitamos muestrear de una categórica (discreta) pero seguir pudiendo hacer backpropagation. **Gumbel-Softmax** es precisamente el truco para esto: en vez de muestrear un índice discreto, se genera una distribución "suave" sobre las K categorías que se puede volver arbitrariamente "dura" (casi un one-hot) bajando una temperatura τ :

$$z_t = \text{Softmax}((l_t + g_t) / \tau), \quad g_t \sim \text{Gumbel}(\theta, 1)$$

Al inicio del entrenamiento τ es alta (la distribución es difusa, fácil de optimizar); se va enfriando gradualmente hasta que z_t se comporta casi como un verdadero one-hot discreto.

(b) El **intérprete (decoder)** ejecuta el programa token por token, de forma recurrente. En vez de condicionar sobre todo z de una vez (como hacía LPN), el decoder de NLI mantiene un **estado interno** s_t que se actualiza en cada paso $t = 1, \dots, T$, consumiendo un token a la vez — literalmente como un intérprete que lee una instrucción, ejecuta, y pasa a la siguiente. Esto tiene una consecuencia importante: el número de "pasos de cómputo" ya no es fijo, sino que **crece con el largo del programa**, permitiendo ejecutar programas más largos que los vistos en entrenamiento (generalización de longitud).

Búsqueda en tiempo de prueba: igual que LPN, NLI también refina su primera propuesta con ascenso de gradiente — pero ahora buscando sobre los embeddings continuos que se relajan (vía Gumbel-Softmax) hacia la secuencia de tokens discretos, no sobre un único vector.

La evidencia de que la discretización es la causa del éxito (no solo una anécdota):

- En la tabla principal, NLI con búsqueda por gradiente logra 99–100% en las tareas de OOD (extrapolar a desplazamientos más largos, extraer sub-funciones, componer primitivas nunca compuestas), mientras que LPN y sus variantes se quedan entre 0% y 29%.
- Un **estudio de ablación** (quitar componentes uno por uno) es muy revelador: quitar la ejecución recurrente o quitar la discreción (del inductor o del intérprete) hace que la precisión OOD caiga de 97% a 1–5%. Es decir, ambos ingredientes — discretización y ejecución recurrente paso-a-paso — son indispensables; ninguno solo basta.
- Un ejemplo muy concreto e ilustrativo: en la tarea de "desplazar una lista a la izquierda n posiciones", entrenando solo con $n \in \{1, \dots, 5\}$, el modelo **descubre por sí solo** un token (llamémoslo "231") que representa "un paso a la izquierda" y otro token ("476") que representa "dos pasos". Para representar un desplazamiento de 4, compone 476 231 ; para uno de 8 (¡nunca visto en entrenamiento!), compone 231 231 231 231 476 476 . Esto es compresión (pocos primitivos) y composicionalidad (reutilización sistemática) emergiendo de forma completamente no supervisada.

2.4 La conjetura central: ¿por qué discretizar ayuda a componer?

Aquí está el razonamiento — en parte explícito en los papers, en parte una síntesis natural que tú mismo puedes hilar:

1. **Conjetura explícita de los autores de NLI:** *"como los programas de NLI pueden recombinar tokens aprendidos de formas distintas y a distintas longitudes, hipotetizamos que su lenguaje soporta generalización combinatoria"* — literalmente llaman a esto una hipótesis en el paper.
2. **La intuición detrás:** un vector continuo $z \in \mathbb{R}^d$ es un único punto en un espacio suave — no tiene "costuras" naturales entre conceptos. Puedes interpolar entre dos programas, pero no hay una noción clara de "combinar la mitad de un programa con la mitad de otro" de forma discreta y reusable. Un conjunto pequeño de **símbolos discretos**, en cambio, funciona como un vocabulario: nuevas "oraciones" (programas) se arman recombinando un número finito y fijo de "palabras" (tokens) en distintos órdenes y longitudes — exactamente como el lenguaje humano, o como ya señalaba el propio paper de VQ-VAE: "el lenguaje es inherentemente discreto... las representaciones discretas son un ajuste natural para el razonamiento complejo, la planificación y el aprendizaje predictivo."
3. **Pero el ablation deja claro que discretizar solo no basta.** También hace falta la ejecución **recurrente, paso a paso**: si el decoder consumiera todo el programa de una vez (como en LPN), no habría ninguna presión para que cada token represente una sub-operación reusable e independiente — el modelo podría volver a "memorizar" combinaciones completas. Al forzar que cada token se procese uno a la vez, actualizando un estado compartido, el modelo *tiene* que aprender primitivos que tengan sentido de forma aislada y compongan bien secuencialmente.

2.5 El cierre del círculo: NLI conecta explícitamente con VQ-VAE

Aquí hay un puente directo y textual entre las dos partes de este apunte, que el propio paper de NLI hace explícito. Al justificar por qué eligieron Gumbel-Softmax y no el mecanismo de VQ-VAE (codebook + straight-through estimator) que estudiaste la semana pasada, los autores escriben que **cualquier relajación diferenciable de un muestreo discreto podría funcionar**, pero prefirieron Gumbel-Softmax por estabilidad de entrenamiento, señalando que el straight-through estimator de VQ-VAE es conocido por sufrir de **gradientes sesgados** y, sobre todo, de "**codebook collapse**" — cuando muchas entradas del codebook simplemente nunca se usan (un fenómeno primo hermano del posterior collapse de la Parte 1: en vez de que el latente colapse hacia el prior completo, colapsa hacia usar solo un subconjunto minúsculo del vocabulario disponible).

Esto conecta ambas partes de tu repaso con un hilo común: **en ambos dominios (VAEs generativos y síntesis de programas), el diseño del cuello de botella latente — continuo vs. discreto, con o sin presión de KL, con o sin mecanismo de cuantización — determina si el modelo efectivamente usa su espacio latente o encuentra un atajo perezoso para ignorarlo (ya sea colapsando al prior, o colapsando a un vocabulario degenerado).**

Resumen ejecutivo (para repasar en 2 minutos)

	Parte 1: VAEs	Parte 2: Síntesis de programas
El problema	Posterior collapse: $q(z x) = p(z)$, el latente se vuelve inútil	Falta de generalización composicional: el modelo memoriza en vez de recombinar primitivos
Explicación teórica 1	Transición de fase: hay un umbral de ruido σ'^2 tras el cual colapsar es óptimo (Li et al.)	LPN usa un z continuo monolítico: no hay unidades discretas que recombinar
Explicación teórica 2	No-identificabilidad: colapso $\Leftrightarrow p(x z)$ no depende de z (Wang, Blei & Cunningham)	El ablation de NLI muestra que discreción + ejecución recurrente son ambas necesarias
La solución con discretización	VQ-VAE: codebook + KL constante ($\log \kappa$) $\rightarrow$ sin presión hacia el colapso	NLI: tokens discretos + intérprete recurrente $\rightarrow$ primitivos reusables y longitud variable
Evidencia empírica	VQ-VAE mantiene latentes informativos incluso con decoders autorregresivos muy potentes	NLI pasa de ~0–29% a 91–100% en tareas OOD composicionales frente a LPN

Para profundizar mañana

Algunas preguntas que quedaron abiertas en los propios papers y que podrías explorar:

- El paper de la transición de fase deja como trabajo futuro estudiar **cómo colapsa cada dimensión latente en distinto momento** (colapso secuencial, no todas a la vez) — ligado a las direcciones principales de los datos.
- El paper de no-identificabilidad admite que su solución (LIDVAE) es más costosa computacionalmente ($O(k \cdot p^2)$) en vez de $O(k \cdot p)$) — vale la pena entender ese trade-off.
- NLI señala como limitación que su intérprete actual no soporta primitivos parametrizados (como `add(k)` con `k` variable) ni ramas condicionales — una dirección natural de extensión.